

Лабораторні роботи (60 балів)

1. Розробка ієрархічної структури веб-сайту. Вибір теми курсового проекту.

Необхідно:

- вибрати та узгодити з викладачем тематику та зміст сайту (який у подальшому буде темою курсового проекту);
- розробити структуру навігації та дизайну сайту.

Окрім того, до лабораторної роботи необхідно докласти ескіз головної сторінки сайту та схему навігації.

При виборі теми і структури сайту керуйтеся кількома вказівками:

- сайт має містити матеріали різного типу, як текст, так і зображення, відео, і т.п.
- сайт має передбачати хоча б кілька способів взаємодії з користувачем: реєстрацію (і авторизацію), коментарі, завантаження файлів (на і з сайту), заповнення форм тощо;
- хоча б деякі сторінки мають містити динамічну інформацію, що змінюється в залежності від дій користувача;
- сайт має мати засоби керування для адміністратора, наприклад управління користувачами і/або контентом тощо.

2. Принципи семантичного дизайну та розмітки. Каскадні таблиці стилів. Таблична та сіткова верстка. Box model.

Необхідно реалізувати:

- семантичну модель кількох (3-4) сторінок сайту за допомогою HTML-розмітки, в тому числі:
 - навігацію між цими сторінками за допомогою гіперпосилань;
 - принаймні 1 таблицю (блок тегів `<table>`) для відображення довільної інформації;
 - принаймні 1 форму (блок тегів `<form>`);
 - принаймні 3 різних типи елементів вводу (тегу `<input>`, `<textarea>`, `<select>` тощо);
 - принаймні 2 різних списки елементів (маркований, нумерований тощо);
 - принаймні 3 плаваючі елементи з різним позиціонуванням;
 - принаймні 5 різних графічних елементи, принаймні 1 з яких має карту посилань;
- стильове оформлення принаймні 1 сторінки у вигляді зовнішнього файлу стилів, у тому числі:
 - принаймні по 1 разу використати усі типи селекторів (селектори класу, ідентифікатора, псевдоелемента та псевдокласу);
 - використати принаймні 1 комбінатор;
 - використати принаймні 1 селектор із атрибутом;
 - вирівнювання всіх елементів сторінки виконати виключно засобами box model.

Додаткові бали будуть нараховані за:

- реалізацію підтримки засобів доступності для людей з обмеженими можливостями (ARIA);
- реалізацію адаптивного дизайну (наприклад, через медіа-запити);
- реалізацію графічних елементів чи анімації засобами CSS;
- грамотне використання flexbox та css grid.

Необхідно знати:

- що таке HTML і об'єктна модель документа (DOM);
- структуру розмітки типової веб-сторінки;
- базові елементи розмітки (теги і атрибути);
- що таке каскадні таблиці стилів (CSS) і як відбувається рендеринг веб-сторінки;
- базові елементи CSS (селектори, атрибути і їх допустимі значення);
- що таке box model, інлайнні та блочні елементи та різницю між ними;
- що таке таблична та сіткова верстка.

В рамках цієї лабораторної роботи забороняється використовувати будь-які готові фреймворки чи бібліотеки для розмітки та стилів.

3. Програмування на стороні клієнта. JavaScript. Маніпуляції DOM та події.

Необхідно реалізувати:

- принаймні 1 сторінку, що містить елементи для взаємодії із користувачем (поля вводу, кнопки тощо);
- зовнішній скрипт, що виконує взаємодію з користувачем:
 - приймає та обробляє дані на сторінці (наприклад, зчитує дані з полів вводу логіну й пароля та порівнює їх із наперед заданими в тілі самого скрипту);
 - модифікує сторінку без її перезавантаження (наприклад, виводить динамічно сформоване повідомлення про помилку у заданому місці сторінки);
- принаймні 2 різних випадки виконання скрипту як реакцію на деяку викликану користувачем подію (натискання кнопки, зміну фокусу тощо; наприклад, виконання скрипту з попереднього пункту по натисканню кнопки і перевірку формату введеного логіну при втраті фокусу полем вводу).

Додаткові бали будуть нараховані за реалізацію динамічного виконання скриптів у залежності від виборів користувача як реакцію на ту саму подію (наприклад, виконання різних функцій при натисканні кнопки в залежності від вмісту деякого поля) при умові його реалізації виключно за допомогою системи подій (а не, наприклад, умовного оператора).

Необхідно знати:

- що таке ECMAScript, JavaScript та TypeScript, та як вони пов'язані;
- що таке асинхронні та відкладені скрипти;
- базові типи даних та функції JavaScript;
- базові функції для маніпуляції DOM;
- визначення об'єктів та масивів у JavaScript;
- типи еквівалентності та правила неявного приведення типів;
- що таке події та як створювати функції-обробники для них.

4. Програмування на стороні клієнта. JavaScript. React та віртуальна DOM. AJAX та асинхронне програмування.

Необхідно реалізувати:

- React-сторінку і набір компонент, що реалізують відображення віджета з погодою на кілька днів (або іншого за погодженням із викладачем);
- асинхронне отримання даних (про погоду), що виконує AJAX-запит (за допомогою fetch чи XMLHttpRequest) на зовнішній сервер (в Інтернеті) та отримує від нього відповідь (наприклад, у форматі JSON);
- блокування інтерфейсу користувача для унеможливлення кількох одночасних запитів (за допомогою приховування елементів UI чи/або відображення тимчасових елементів-заглушок, “лоадерів”/прогрес-барів);
- обробку помилок комунікації із зовнішнім сервером (неправильних даних запиту, таймаутів тощо).

Додаткові бали будуть нараховані за:

- виконання запитів не (тільки) у момент завантаження сторінки, а динамічно, в залежності від дій користувача та в залежності від уведених даних (наприклад, погоду в залежності від обраного міста);
- відображення великого масиву даних шляхом стрімінгу: динамічного підвантаження та вивантаження даних, що не контролюється користувачем безпосередньо (наприклад, прогнозу на наступні дні при скролінгу у віджеті);
- використання Geolocation API;
- складні у реалізації компоненти React, використання Redux.

Необхідно знати:

- що таке AJAX, CORS, XMLHttpRequest та fetch, чим вони відрізняються;
- що таке JSON, XML та які ще формати можуть передаватися серверами за допомогою AJAX;
- базові функції та події для виконання запитів на зовнішні та внутрішні сервери;
- базові функції для роботи з асинхронністю.

Для отримання даних рекомендується використовувати один із наступних сервісів у відкритому доступі:

- <https://openweathermap.org/api> — дані про погоду;
- <https://data.worldbank.org/> — світові економічні дані;
- <https://www.opendatanetwork.com/> — каталог відкритих ресурсів з даними.

5. Взаємодія клієнта і сервера. Налаштування локального сервера для React.

Необхідно реалізувати систему реєстрації та авторизації користувачів виконавши:

- модифікацію принаймні 2-х сторінок з попередніх робіт, що використовує React для генерації частини вмісту сторінки на стороні серверу, серед іншого:
 - частина контенту має бути динамічною, тобто залежати від зовнішніх чинників (стану cookies, переданих у GET чи POST запиті даних, часу тощо; наприклад, темні кольори дизайну в нічний час);

- частина контенту має бути видобута з файлової системи (текстових файлів, бази даних тощо; наприклад, коментарі користувачів мають зберігатися у текстовому файлі на сервері);
- модифікацію принаймні 1-ї сторінки з попередніх робіт, що передає інформацію на сервер у вигляді GET чи POST запиту, аби ця інформація оброблялася там (наприклад, передавати номер сторінки з коментарями користувачів, при відображенні їх порціями);
- демонстрацію виконаних змін на локальному комп'ютері.

Додаткові бали будуть нараховані за:

- грамотну реалізацію механізму реєстрації та авторизації користувачів, включаючи передачу та зберігання паролів у зашифрованому вигляді;
- розміщення сайту не на локальному, а на віддаленому комп'ютері, доступному з мережі (наприклад, на віддаленому тестовому комп'ютері з запущеним CLI SAPI з видимим зовні IP — це не обов'язково має бути повноцінне розміщення на віддаленому хостингу чи у хмарі);
- використання баз даних.

Необхідно знати:

- що таке вебсервер;
- спеціальні змінні для доступу до персистентних елементів у JavaScript;
- різницю між GET та POST запитами;
- способи збереження персистентного стану між клієнтом і сервером.

6. Програмування на стороні сервера. REST API та обмін даними між клієнтом і сервером.

Необхідно реалізувати мінімалістичне REST API для свого сайту, серед іншого:

- принаймні 3 файли, що генерують не повноцінну веб-сторінку, а частину веб-сторінки, форматовані дані (JSON, XML тощо) чи текстові дані;
- систему переадресації (роутингу) засобами React;
- принаймні 1 сторінку, що звертається до цих API-ендпоінтів, використовуючи AJAX, подібно тому, як це було в лабораторній роботі №4 для зовнішнього сервера (наприклад, завантажувати коментарі порціями по натисканню на кнопку, що відправляє запит до API, який повертає вже згенерований шматок HTML-розмітки з новими коментарями, чи їх же у вигляді JSON-об'єкту, та скрипт для відображення їх на стороні клієнта).

Додаткові бали будуть нараховані за документування API за допомогою [Swagger](#).

Необхідно знати:

- що таке API, REST, RESTful API;
- в чому різниця між протоколами зі зберіганням і без зберігання стану;
- що таке SOAP та RPC;
- які бувають архітектури розподілених систем;
- як реалізовується роутинг веб-запиту.

В лабораторних роботах 5 та 6 замість JavaScript можна використовувати іншу мову програмування, але лише за *попередньою згодою* викладача.